LAPORAN AKHIR PRAKTIKUM

PEMOGRAMAN BERBASIS OBJEK



Dosen Pengampu: Nurfiah, S.ST, M.Kom

Asisten Praktikum: M. Dawi Syauqi

Disusun Oleh:

Aliyatar Rafi Ahmad 251153301

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

1.1 Praktikum 1 — Class, Object, Constructor, dan Method

Tujuan: Memahami konsep dasar PBO serta menerapkannya dalam program perbankan mini menggunakan Java.

Implementasi: Program menggunakan class Rekening untuk menyimpan nomor rekening, nama pemilik, dan saldo. Constructor digunakan untuk menginisialisasi data rekening, sedangkan method setorTunai(), tarikTunai(), dan cekInformasi() digunakan untuk mengelola transaksi dan menampilkan informasi rekening.

```java
package pekan1;
public class Rekening {
    String nomorRekening;
    String namaPemilik;
    double saldo;

    public Rekening(String nomor,String nama, double saldoAwal) {
        nomorRekening = nomor;
        namaPemilik = nama;
        saldo = saldoAwal;
        System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat dengan saldo Rp" + saldo);
    }

    public void setorTunai(double nominal) {
        if (nominal > 10000) {
            saldo += nominal;
            System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
        } else {
            System.out.println("Gagal: Nominal setor harus lebih dari 10000!");
        }
    }

    public void tarikTunai(double nominal) {
        if (nominal >= 10000) {
            if ( nominal > saldo) {
                System.out.println(" Transaksi gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
            }
            else{saldo -= nominal;
                System.out.println("Tarik tunai" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
            }
        } else {
            System.out.println("Gagal: Nominal tarik saldo adalah 10000!");
        }
    }

    public void cekInformasi() {
        System.out.println("--- INFO REKENIG ---");
        System.out.println("No. Rekening : " + nomorRekening);
        System.out.println("Nama Pemilik : " + namaPemilik);
        System.out.println("Saldo Akhir  :Rp" + saldo);
        System.out.println("---------------------");
    }
    public String getNomorRekening() {
        return nomorRekening;
    }
}
```

Pada class Main, digunakan ArrayList untuk menyimpan beberapa rekening dan variabel akunAktif untuk menentukan rekening yang sedang digunakan. Program juga menyediakan menu untuk membuka rekening, melakukan transaksi, melihat informasi, dan mengganti akun.

```java
package pekan1;
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        ArrayList<Rekening> daftarRekening = new ArrayList<>();
        Rekening akunAktif = null; //Objek belum diinstalisasi (null)
        boolean isRunning = true;

        System.out.println("=== SISTEM PERBANKAN MINI ===");

        while (isRunning) {
            System.out.println("\nMenu Utama");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik Tunai");
            System.out.println("4. Cek Informasi Rekening");
            System.out.println("5. Ganti Akun");
            System.out.println("0. Keluar");
            System.out.print("Pilih Menu: ");

            int pilihan = input.nextInt();
            input.nextLine(); // Membersihkan buffer enter

            switch (pilihan) {
            case 1:
                System.out.print("Masukkan No Rekening: ");
                String no = input.nextLine();
                System.out.print("Masukkan Nama Pemilik: ");
                String nama = input.nextLine();
                System.out.print("Masukkan Saldo Awal: ");
                double saldo = input.nextDouble();

                // instansiasi Object / menjalankan konstruktor
                Rekening rekeningBaru = new Rekening(no, nama, saldo);
                daftarRekening.add(rekeningBaru);
                akunAktif = rekeningBaru;
                break;

            case 2:
                if (akunAktif == null) {
                    System.out.println("Error: Mohon maaf, Anda belum memiliki nomor rekening!");
                } else {
                    System.out.print("Masukkan nominal setor: ");
                    double setor = input.nextDouble();
                    akunAktif.setorTunai(setor); // Memanggil Behavior / Method
                }
                break;

            case 3:
                if (akunAktif == null) {
                    System.out.println("Error: Mohon maaf, Anda belum memiliki nomor rekening!.");
                } else {
                    System.out.println("Masukkan nominal tarik: ");
                    double tarik = input.nextDouble();
                    akunAktif.tarikTunai(tarik);
                }

                break;

            case 4:
                if (akunAktif == null) {
                    System.out.println("Error: Anda belum membuka rekening!");
                } else {
                    akunAktif.cekInformasi();
                }
                break;

            case 5:
                System.out.print("Masukkan No Rekening: ");
                String nomorCari = input.nextLine();

                Rekening akunDitemukan = null;

                for (Rekening rekening : daftarRekening) {
                    if (rekening.nomorRekening.equals(nomorCari)) {
                        akunDitemukan = rekening;
                        break;
                    }
                }

                if (akunDitemukan != null) {
                    akunAktif = akunDitemukan;
                    System.out.println("Berhasil mengganti akun.");
                } else {
                    System.out.println("Rekening tidak ditemukan!");
                }
                break;

            case 0:
                isRunning = false;
                System.out.println("System ditutup. Terima Kasih!");
                break;

            default:
                System.out.println("Pilihan tidak valid");
            }
        }
        input.close();
    }
}
```

Latihan/Tugas: Menambahkan fitur tarik tunai dengan validasi nominal minimal Rp10.000 dan saldo yang mencukupi. Selain itu, program dikembangkan menggunakan

ArrayList<Rekening> agar dapat menyimpan banyak rekening, serta menyediakan fitur ganti akun berdasarkan nomor rekening.

Note: Kode program diatas sudah termasuk implementasi dari tugas.

Kesimpulan: Praktikum ini menerapkan konsep class, object, constructor, dan method, serta mempelajari manipulasi saldo dan pengelolaan beberapa objek rekening.

1.2 Praktikum 2 — Relasi Antar Objek dan Asosiasi

Tujuan: Memahami relasi antarobjek dengan konsep asosiasi one-to-many serta mengelola kumpulan objek menggunakan ArrayList.

Implementasi: Program dikembangkan dengan menambahkan class Transaksi yang memiliki atribut idTransaksi, jenis, dan nominal.

```java
package pekan2;

public class Transaksi {
    String idTransaksi;
    String jenis;
    double nominal;

    //construktor
    public Transaksi(String id, String jenis, double nominal) {
        this.idTransaksi = id;
        this.jenis = jenis;
        this.nominal = nominal;
    }
    public void cetakDetail() {
        System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal);
    }
}
```

Pada class Rekening, ditambahkan ArrayList<Transaksi> untuk menyimpan riwayat transaksi. Setoran dicatat sebagai Kredit, sedangkan penarikan dicatat sebagai Debit. Method cetakMutasi() digunakan untuk menampilkan riwayat transaksi melalui perulangan for-each. Menu cetak mutasi juga ditambahkan pada class Main.

```java
package pekan2;
import java.util.ArrayList;
public class Rekening {
    String nomorRekening;
    String namaPemilik;
    double saldo;

    //Implementasi Asosiasi (1-to-many)
    ArrayList<Transaksi> riwayatTransaksi;

    public Rekening(String nomor,String nama, double saldoAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        // Wajib menginisialisasi Arraylist di dalam construktor agar tidak nullPointerException
        this.riwayatTransaksi = new ArrayList<>();

        System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat dengan saldo Rp" + saldo);
    }
```

```java
public void setorTunai(double nominal) {
    if (nominal <= 500000) {
        saldo += nominal;
        // Merekam riwayat (Pembuatan objek Transaksi di dalam method)
        String idTrx = "TRX-S-" + System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
        riwayatTransaksi.add(trxBaru);

        System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
    } else {
        System.out.println("Gagal: Nominal setor maksimal Rp.500.000!");
    }
}
```

Latihan/Tugas:

1. Integrasi riwayat penarikan: Menambahkan pencatatan transaksi bertipe Debit dengan ID berawalan TRX-T- setiap kali penarikan berhasil.

```java
public void tarikTunai(double nominal) {
    if (nominal >= 10000) {
        if ( nominal > saldo) {
        System.out.println(" Transaksi gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
        }
        else{saldo -= nominal;
        // Merekam riwayat (Pembuatan objek Transaksi di dalam method)
        String idTrx = "TRX-T-" + System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
        riwayatTransaksi.add(trxBaru);
            System.out.println("Tarik tunai" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
        }
    } else {
        System.out.println("Gagal: Nominal tarik saldo adalah 10000!");
    }
}
```

2. Fitur cetak mutasi: Membuat method cetakMutasi() menggunakan perulangan for-each untuk menampilkan seluruh riwayat transaksi. Jika riwayat masih kosong, program menampilkan pesan bahwa belum ada transaksi. Fitur ini kemudian dihubungkan dengan menu nomor 6 pada class Main.

```java
case 6:
    if (akunAktif == null) {
        System.out.println("Tidak ada rekening yang tersedia");
    } else {
        akunAktif.cetakMutasi();
    }
    break;
```

Kesimpulan: Praktikum ini menerapkan relasi one-to-many, yaitu satu rekening dapat memiliki banyak transaksi, serta memungkinkan program mencatat dan menampilkan riwayat transaksi.

1.3 Praktikum 3 — Keamanan & Integritas Data (Enkapsulasi)

Tujuan: Memahami konsep information hiding, penggunaan access modifier private, protected, dan public, serta penerapan getter, setter, dan autentikasi PIN untuk menjaga keamanan data.

Implementasi: Pada class Transaksi, atribut diubah menjadi private agar tidak dapat diakses langsung dari luar class. Getter disediakan untuk membaca data yang diperlukan.

```java
package pekan3;

public class Transaksi {
    // 1. Mengubah semua atribut menjadi private
    private String idTransaksi;
    private String jenis;
    private double nominal;

    //construktor
    public Transaksi(String id, String jenis, double nominal) {
        this.idTransaksi = id;
        this.jenis = jenis;
        this.nominal = nominal;
    }
```

Pada class Rekening, atribut diamankan menggunakan access modifier yang sesuai, serta ditambahkan PIN dan method otentikasi() untuk memeriksa kebenaran PIN.

```java
package pekan3;
import java.util.ArrayList;
public class Rekening {
    // 1. Mengunci atribut dengan 'private'
    private String nomorRekening;
    private String namaPemilik;
    private double saldo;
    private String pin; // Data sensitif

    ArrayList<Transaksi> riwayatTransaksi;

    // 2. Modifikasi Construktor untuk menerima PIN awal
    public Rekening(String nomor,String nama, double saldoAwal, String pinAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        // Validasi PIN di dalam Construktor
        if (pinAwal.length() == 6) {
            this.pin = pinAwal;
        } else {
            System.out.println("Peringatan: PIN harus 6 digit! Menggunakan pin default 123456");
            this.pin = "123456";
        }

        this.riwayatTransaksi = new ArrayList<>();
        System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat dengan saldo Rp" + saldo);
    }
    // 3. Getter untuk atribut yang diizinkan oleh publik
    public String getNomorRekening() { return nomorRekening; }
    public String getNamaPemilik() { return namaPemilik; }

    // 4. Method Otentikasi Internal (Validasi Enkapsulasi)
    public boolean otentikasi(String inputPin) {
        return this.pin.equals(inputPin);
    }
```

Latihan/Tugas — Mengamankan CLI:

1. Menu 1 (Buka Rekening): Meminta pengguna memasukkan PIN berupa enam digit sebelum objek rekening dibuat.

```java
case 1:
    System.out.print("Masukkan No Rekening: ");
    String no = input.nextLine();
    System.out.print("Masukkan Nama Pemilik: ");
    String nama = input.nextLine();
    System.out.print("Masukkan Saldo Awal: ");
    double saldo = input.nextDouble();
    input.nextLine();
    System.out.print("Masukkan Pin Anda: ");
    String pin = input.nextLine();

    // instansiasi Object / menjalankan konstruktor
    Rekening rekeningBaru = new Rekening(no, nama, saldo, pin);
    daftarRekening.add(rekeningBaru);
    akunAktif = rekeningBaru;
    break;
```

2. Menu 3 (Tarik Tunai): Meminta PIN dan menjalankan penarikan hanya jika autentikasi berhasil. Jika PIN salah, program menampilkan pesan "Akses Ditolak: PIN yang Anda masukkan salah!".

```
case 3:
    if (akunAktif == null) {
        System.out.println("Error: Mohon maaf, Anda belum memiliki nomor rekening!.");
    } else {
        System.out.println("Masukkan Pin Anda: ");
        String pinTarik = input.nextLine();

        if (akunAktif.otentikasi(pinTarik)) {
            System.out.println("Masukkan nominal tarik: ");
            double tarik = input.nextDouble();
            input.nextLine();
            akunAktif.tarikTunai(tarik);
        } else
        System.out.println("Akses ditolak! Pin yang anda masukkan salah");
    }
    break;
```

3. Menu 6 (Cetak Mutasi): Meminta PIN sebelum menampilkan riwayat transaksi. Jika PIN salah, akses ditolak dan riwayat tidak ditampilkan.

```
case 6:
    if (akunAktif == null) {
        System.out.println("Tidak ada rekening yang tersedia");
    } else {
        System.out.println("Masukkan PIN Anda");
        String pinMutasi = input.nextLine();

        if (akunAktif.otentikasi(pinMutasi)) {
            akunAktif.cetakMutasi();
        } else {
            System.out.println("Akses Ditolak! PIN yang Anda masukkan salah!");
        }
    }
    break;
```

Kesimpulan: Praktikum ini menerapkan enkapsulasi untuk membatasi akses langsung terhadap atribut serta menambahkan autentikasi PIN agar transaksi dan riwayat rekening lebih terlindungi.

1.4 Praktikum 4 — Inheritance

Tujuan: Memahami konsep inheritance atau pewarisan, penggunaan extends dan super(), serta penerapan upcasting dan downcasting pada Java.

Implementasi: Class Rekening dijadikan superclass yang mewariskan atribut dan method kepada dua subclass, yaitu RekeningTabungan dan RekeningGiro.

```
// bagian atas dari class Rekening.java
private String nomorRekening;
private String namaPemilik;
private String pin; // Data sensitif

// Gunakan protected agar Subclass bisa mengaksesnya langsung
protected double saldo;
protected ArrayList<Transaksi> riwayatTransaksi;
```

RekeningTabungan memiliki atribut tambahan sukuBunga serta method tambahBungaAkhirBulan().

```java
package pekan4;

public class RekeningTabungan extends Rekening {

    private double sukuBunga;

    public RekeningTabungan(String nomor, String nama, double saldoAwal, String pinAwal, double sukuBunga) {
        // super() memanggil construktor kelas induk (Rekening). WAJIB berasa di baris pertama
        super(nomor, nama, saldoAwal, pinAwal);
        this.sukuBunga = sukuBunga;
    }
    public void tambahBungaAkhirBulan() {
        double nominalBunga = saldo * (sukuBunga / 100);
        saldo += nominalBunga;

        String idTrx = "TRX-B-" + System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi(idTrx, "Bunga", nominalBunga));
        System.out.println("Bunga " + sukuBunga + "% berhasil ditambahkan: Rp" + nominalBunga);
    }
}
```

Sedangkan RekeningGiro memiliki atribut batasOverdraft. Konstruktor kedua subclass menggunakan super() untuk memanggil konstruktor dari class Rekening.

```java
package pekan4;

public class RekeningGiro extends Rekening {
    private double batasOverdraft;

    public RekeningGiro(String nomor, String nama, double saldoAwal, String pinAwal, double batasOverdraft) {
        super(nomor, nama, saldoAwal, pinAwal);
        this.batasOverdraft = batasOverdraft;
    }
    public double getBatasOverdraft() {
        return batasOverdraft;
    }
}
```

Latihan/Tugas:

1. Menu 1 (Buka Rekening): Pengguna dapat memilih jenis rekening Tabungan atau Giro. Objek yang dibuat disimpan dalam variabel bertipe Rekening, lalu dimasukkan ke daftarRekening. Hal ini menerapkan konsep upcasting.

```java
case 1:
    System.out.print("Masukkan No Rekening: ");
    String no = input.nextLine();
    System.out.print("Masukkan Nama Pemilik: ");
    String nama = input.nextLine();
    System.out.print("Masukkan Saldo Awal: ");
    double saldo = input.nextDouble();
    input.nextLine();
    System.out.print("Masukkan Pin Anda (6 digit) : ");
    String pin = input.nextLine();
    System.out.println("Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis | " );
    int pilihanProduk = input.nextInt();
    // Variabel untuk menampung objek rekening
    Rekening rekeningBaru = null;

    if (pilihanProduk == 1) {
        System.out.print("Masukkan Suku Bunga (%) : ");
        double sukuBunga = input.nextDouble();
        rekeningBaru = new RekeningTabungan(no, nama, saldo, pin, sukuBunga);
    } else if (pilihanProduk == 2) {
        System.out.print("Masukkan Batas Overdraft : ");
        double batasOverdraft = input.nextDouble();
        rekeningBaru = new RekeningGiro(no, nama, saldo, pin, batasOverdraft);
    } else {
        System.out.println("Pilihan produk tidak valid.");
        break;
    }
    daftarRekening.add(rekeningBaru);
    akunAktif = rekeningBaru;
    break;
```

2. Menu 7 (Tambah Bunga): Program menggunakan instanceof untuk memeriksa apakah rekening aktif merupakan RekeningTabungan. Jika iya, objek di-downcasting ke RekeningTabungan agar method tambahBungaAkhirBulan() dapat dijalankan. Jika rekening bukan tabungan, program menampilkan pesan bahwa fitur bunga hanya berlaku untuk rekening tabungan.

```
case 7:
    if (akunAktif instanceof RekeningTabungan) {
        RekeningTabungan tabungan =
                (RekeningTabungan) akunAktif;
        tabungan.tambahBungaAkhirBulan();
    } else {
        System.out.println("Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan");
    }
    break;
```

Kesimpulan: Praktikum ini menerapkan inheritance untuk menggunakan kembali atribut dan method dari class induk. Dengan demikian, setiap jenis rekening dapat memiliki fitur khusus tanpa perlu membuat ulang seluruh kode dari class Rekening.

1.5 Praktikum Pekan 5 — Polimorfisme

Tujuan: Memahami konsep polimorfisme melalui method overriding dan dynamic dispatch pada Java, sehingga method yang dijalankan dapat menyesuaikan jenis objeknya.

Implementasi: Pada class Rekening, method tarikTunai() direfaktor agar mengembalikan nilai boolean. Method ini mengembalikan true jika penarikan berhasil dan false jika gagal. Nilai tersebut dapat digunakan untuk menentukan apakah transaksi berikutnya boleh dijalankan.

```
public boolean tarikTunai(double nominal) {
    if (nominal <= 10000) {
        System.out.println("Gagal: Nominal tarik harus lebih dari 10000");
        return false;
    } else if (nominal > saldo) {
        System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo anda: Rp." + saldo);
        return false;
    } else {
        saldo -= nominal;

        String idTrx = "TRX-T-" + System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi(idTrx, "Debit", nominal));
        return true;
    }
}
```

Selanjutnya, pada class RekeningGiro, method tarikTunai() di-override untuk menerapkan aturan penarikan khusus rekening Giro. Penarikan dapat dilakukan melebihi saldo selama jumlahnya tidak melewati batas overdraft. Jika berhasil, saldo diperbarui dan transaksi dicatat dalam riwayat.

```java
package pekan5;

public class RekeningGiro extends Rekening {
    private double batasOverdraft;

    public RekeningGiro(String nomor, String nama, double saldoAwal, String pinAwal, double batasOverdraft) {
        super(nomor, nama, saldoAwal, pinAwal);
        this.batasOverdraft = batasOverdraft;
    }

    // IMPLEMENTASI POLIMORFISME (OVERRIDING)
    @Override
    public boolean tarikTunai(double nominal) {
        if (nominal <= 0) {
            System.out.println("Gagal: Nominal tarik Giro harus lebih dari 0!");
            return false;
        }
        // Logika Khusus Giro: Boleh ditarik asalkan nominal tidak melebihi (saldo + batasOverdraft)
        else if (nominal > (saldo + batasOverdraft)) {
            System.out.println("Transaksi Gagal: Melebihi batas overdraft! Maksimal penarikan: Rp" + (saldo + batasOverdraft));
            return false;
        }
        else {
            saldo -= nominal; // Saldo bisa bernilai negatif di sini (memakan overdraft)

            String idTrx = "TRX-G-" + System.currentTimeMillis();
            riwayatTransaksi.add(new Transaksi(idTrx, "Debit Giro", nominal));

            System.out.println("Tarik tunai Giro Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
            return true;
        }
    }

    public double getBatasOverdraft() {
        return batasOverdraft;
    }
}
```

Latihan/Tugas: Program juga ditambahkan method transfer() pada class Rekening. Method ini memeriksa PIN, kemudian melakukan penarikan dari rekening pengirim. Jika penarikan berhasil, uang disetorkan ke rekening tujuan. Pemanggilan tarikTunai() memungkinkan Java menjalankan method sesuai jenis objek rekening yang digunakan.

```java
public void transfer(Rekening tujuan, double nominal, String pinInput) {
    if (!otentikasi(pinInput)) {
        System.out.println("Transfer gagal: PIN yang Anda masukkan salah!");
        return;
    }

    if (this.tarikTunai(nominal)) {
        tujuan.setorTunai(nominal);
    }
}
```

```java
case 8:
    if (akunAktif == null) {
        System.out.println("Anda belum memiliki rekening aktif!");
    } else {
        System.out.print("Masukkan Nomor Rekening Tujuan: ");
        String nomorTujuan = input.nextLine();

        Rekening akunTujuan = null;

        for (Rekening rekening : daftarRekening) {
            if (rekening != null &&
                rekening.getNomorRekening().equals(nomorTujuan)) {
                akunTujuan = rekening;
                break;
            }
        }

        if (akunTujuan == null) {
            System.out.println("Rekening tujuan tidak ditemukan!");
        } else {
            System.out.print("Masukkan Nominal Transfer: ");
            double nominal = input.nextDouble();
            input.nextLine();

            System.out.print("Masukkan PIN Anda: ");
            String pinTransfer = input.nextLine();

            akunAktif.transfer(akunTujuan, nominal, pinTransfer);
        }
    }
    break;
```

Kesimpulan: Praktikum ini menerapkan polimorfisme melalui overriding serta dynamic dispatch. Refaktor method tarikTunai() menjadi boolean membantu program mengetahui keberhasilan penarikan, sedangkan fitur transfer memungkinkan pemindahan uang antarrekening dengan validasi PIN dan penarikan.

Kesimpulan Praktikum PBO Pekan 1–5

Berdasarkan praktikum pekan 1 sampai 5, program rekening bank dikembangkan secara bertahap dengan menerapkan konsep dasar Pemrograman Berorientasi Objek (PBO) menggunakan Java. Pada pekan 1, dipelajari penggunaan class, object, constructor, dan method untuk membuat fitur dasar rekening, seperti setor tunai dan tarik tunai. Pekan 2 menambahkan pencatatan transaksi dan fitur mutasi rekening menggunakan ArrayList. Pada pekan 3, diterapkan konsep enkapsulasi untuk membatasi akses langsung terhadap atribut serta menambahkan autentikasi PIN demi menjaga keamanan data rekening.

Selanjutnya, pada pekan 4, diterapkan konsep inheritance dengan membuat class RekeningTabungan dan RekeningGiro sebagai turunan dari class Rekening. Kedua jenis rekening tersebut memiliki fitur tambahan sesuai fungsinya. Pada pekan 5, diterapkan konsep polimorfisme melalui method overriding pada fitur tarik tunai, sehingga setiap jenis rekening dapat memiliki aturan penarikan yang berbeda. Selain itu, ditambahkan fitur transfer antarrekening dengan validasi PIN dan pemeriksaan keberhasilan penarikan.

Secara keseluruhan, praktikum ini memberikan pemahaman tentang penerapan konsep PBO untuk membangun program yang terstruktur, aman, dan mudah dikembangkan. Setiap materi saling berkaitan dalam pengembangan sistem rekening bank, mulai dari fitur dasar hingga penerapan pewarisan dan polimorfisme.

Link Repository:

PrakPBO_C_2511533031/src at pekan5 · Atar09/PrakPBO_C_2511533031